Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 8

Project Title: High-Concurrency File Distribution Server

Team Members

Roll Number	Student Name	Signature
2520030231	K.Girish Chandra	
2520030450	P.Venkat	
2520030307	N.V.Karthikeya	

1. Abstract

The High-Concurrency File Distribution Server is an Operating Systems-based project designed to efficiently handle multiple file requests from clients simultaneously. The main objective of the system is to demonstrate how concurrency, process management, multithreading, synchronization, and resource management can be applied to build an efficient file distribution system. The server creates and manages multiple threads or processes to serve concurrent client requests, allowing several users to access or download files at the same time. Thread synchronization mechanisms such as mutex locks and semaphores are used to prevent race conditions and ensure safe access to shared files and resources. The system also demonstrates concepts such as inter-process communication (IPC), scheduling, memory management, and file handling. Proper resource allocation and synchronization help maintain system stability and prevent conflicts when multiple clients access the same resources simultaneously. The project provides a practical implementation of core Operating Systems concepts while demonstrating how concurrent tasks can be managed efficiently. This system can be extended to support large numbers of clients and optimized for better CPU and memory utilization, making it a useful example of applying Operating Systems principles to a real-world server environment.

2. Problem Statement

In a traditional file distribution system, handling a large number of clients simultaneously can lead to slow response times, resource contention, race conditions, and inefficient CPU utilization. When multiple users request or access files at the same time, improper management of processes, threads, memory, and shared resources may cause conflicts or server performance issues. Therefore, there is a need for a High-Concurrency File Distribution Server that can efficiently manage multiple client requests concurrently while ensuring safe and reliable access to files. The proposed system focuses on applying Operating Systems concepts such as multithreading, process management, synchronization, semaphores, mutex locks, inter-process communication, and resource management to handle concurrent file operations. The system aims to improve performance, prevent race conditions, optimize resource utilization, and provide reliable file distribution even when many clients access the server simultaneously.

3. Objectives

1. To implement concurrent file distribution by allowing multiple clients to request and receive files simultaneously.
2. To demonstrate multithreading and process management for efficiently handling multiple client requests.
3. To implement synchronization mechanisms such as mutex locks and semaphores to prevent race conditions and ensure safe access to shared files.
4. To efficiently manage system resources such as CPU, memory, file descriptors, and threads during high-concurrency operations.
5. To demonstrate Inter-Process Communication (IPC) for communication and coordination between processes when required.
6. To improve server performance and responsiveness by efficiently scheduling and managing concurrent tasks.

4. Proposed Methodology

The High-Concurrency File Distribution Server will be developed using Operating Systems concepts to efficiently handle multiple file requests at the same time. The methodology focuses on process management, multithreading, synchronization, resource management, and file handling.

1.Client Request Handling

The server continuously listens for incoming client requests. When a client requests a file, the server accepts the connection and creates a separate thread/process to handle the request.

2.Multithreading / Concurrency

Multiple threads will be used to serve multiple clients simultaneously. This prevents one client's file transfer from blocking other clients and improves overall server responsiveness.

3.File Management

The server maintains a designated directory containing the files available for distribution. When a request is received, the server checks whether the requested file exists and prepares it for transfer

4.Synchronization

Mutex locks and/or semaphores will be used to control access to shared resources. This prevents race conditions when multiple threads attempt to access or modify the same file or server resource.

5.Inter-Process Communication (IPC)

If multiple processes are used, IPC mechanisms such as pipes, shared memory, or message queues can be implemented for communication and coordination between processes.

6.Resource Management

The server will monitor and manage resources such as CPU, memory, threads, file descriptors, and network connections to prevent resource exhaustion during high-concurrency operations.

7.Error and Exception Handling

The system will handle errors such as invalid file requests, unavailable files, interrupted connections, and resource limitations without crashing the server.

8.Performance Evaluation

The server will be tested with different numbers of simultaneous clients. Performance will be evaluated using parameters such as response time, throughput, CPU utilization, memory usage, and number of successfully served requests.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – fork()	Creates child processes to handle multiple client requests concurrently.
Process Identification – getpid() / getppid()	Retrieves process IDs for process identification and management.
Signal Handling – signal() / sigaction()	Registers signal handlers to safely respond to signals.
Signal Sending – kill()	Sends signals from the server process to child processes.
Process Synchronization – wait()	Allows the parent process to wait for child processes to complete.
Process Control – SIGSTOP, SIGCONT, SIGTERM	Provides safe termination and control of server processes.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030231	K.Girish Chandra	Developed the server and handled client connections.
2520030450	P.Venkat	Implemented multithreading and synchronization.
2520030307	N.V.Karthikeya	Implemented file handling, testing, and documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the process communication program.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional Linux-based high-concurrency file distribution server that demonstrates how multiple clients can request and receive files simultaneously using Operating Systems concepts.

1. The server will successfully accept multiple client connections concurrently.
2. Multiple processes/threads will handle client requests simultaneously.
3. Files will be read and distributed successfully to the requested clients.
4. Mutexes and semaphores will prevent race conditions during shared resource access.
5. The server will efficiently manage CPU, memory, and file resources.
6. The system will handle client disconnections and errors without crashing.
7. The terminal output will clearly show client connections, file requests, file transfers, and process/thread activities.
8. Overall, the project will provide practical understanding of Linux process management, multithreading, synchronization, file management, concurrency, and system calls.

9. GitHub Repository

GitHub Repository Link: https://github.com/PusunuruVenkat/Team-8_OSSP

Repository Created:

□ Yes □ No

Faculty Name and Signature

Faculty Name: dr.Y.Harika devi

Signature:

Remarks:

Faculty Use

Project Approved:

□ Yes □ No

Date:

Faculty Signature: